

Getting Started with Local Models (Gemma 4, Llama, and More)

Lucas Soares

October 2026

Methodology Notes

1. Presentation Block

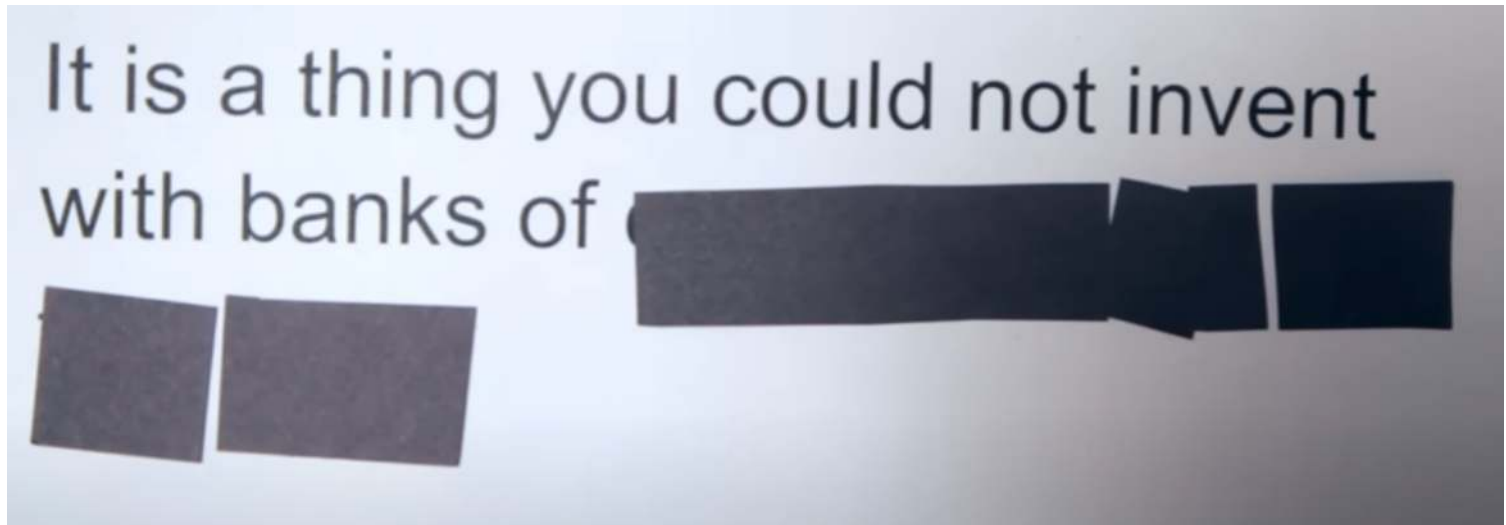
1. Notebook Demo

1. Quick Q&A + Summary

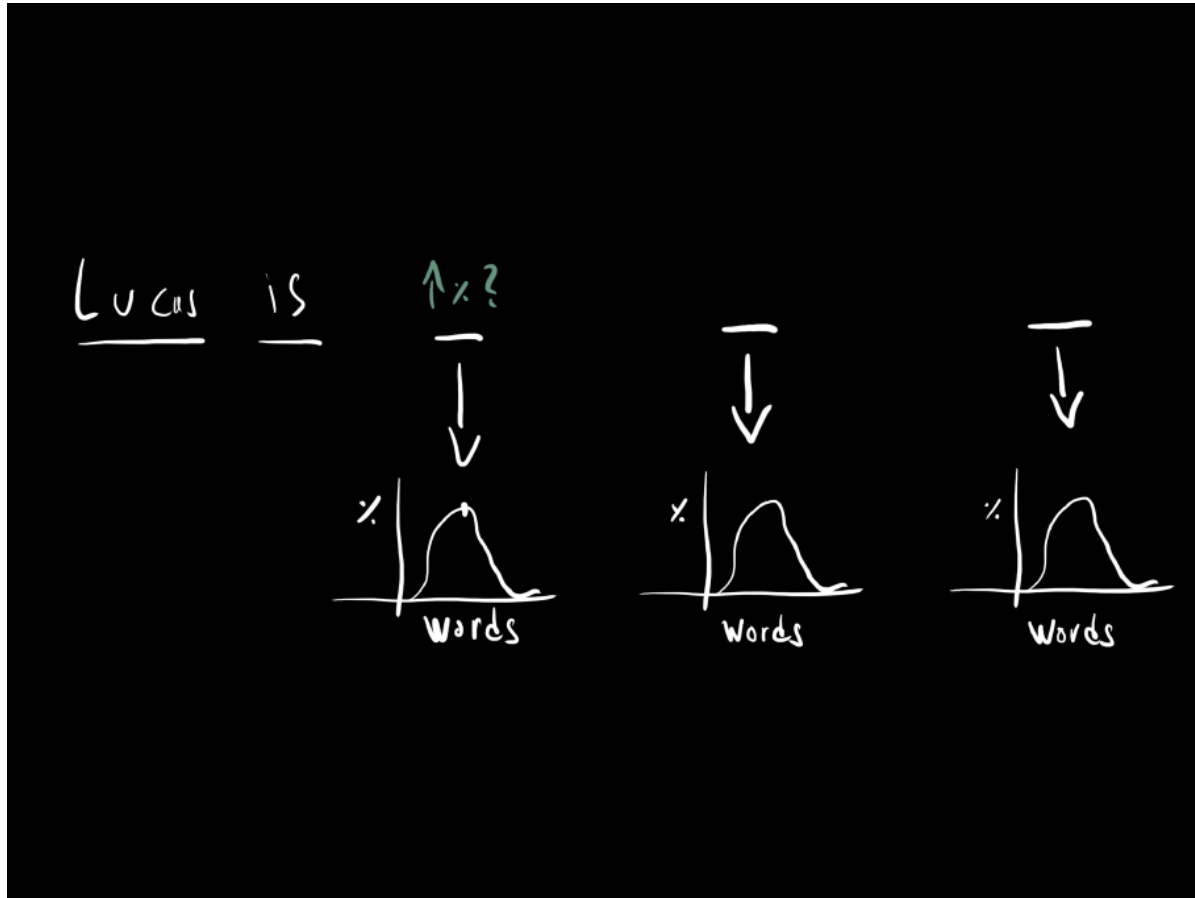
1. Optional Exercise During Q&A

1. Repeat

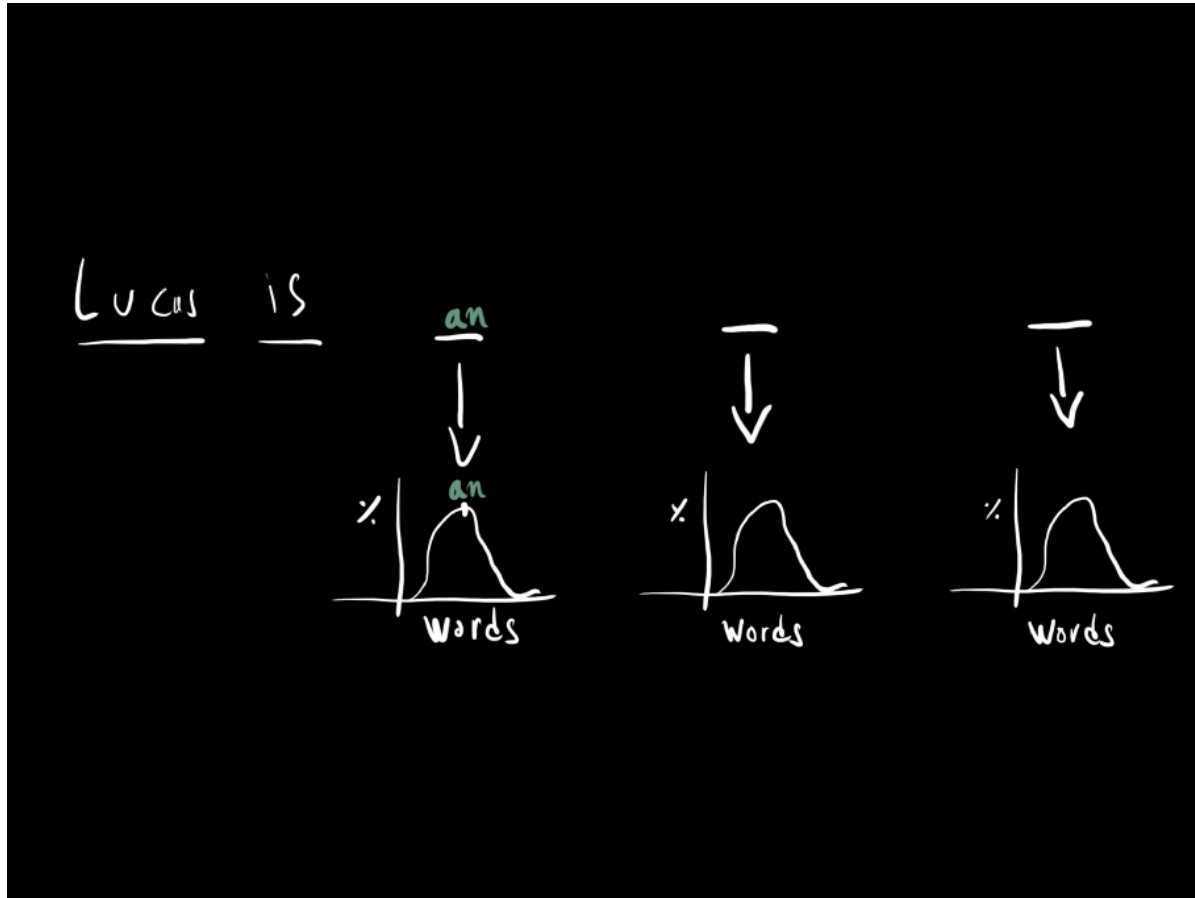
LLMs Predict the Next Word



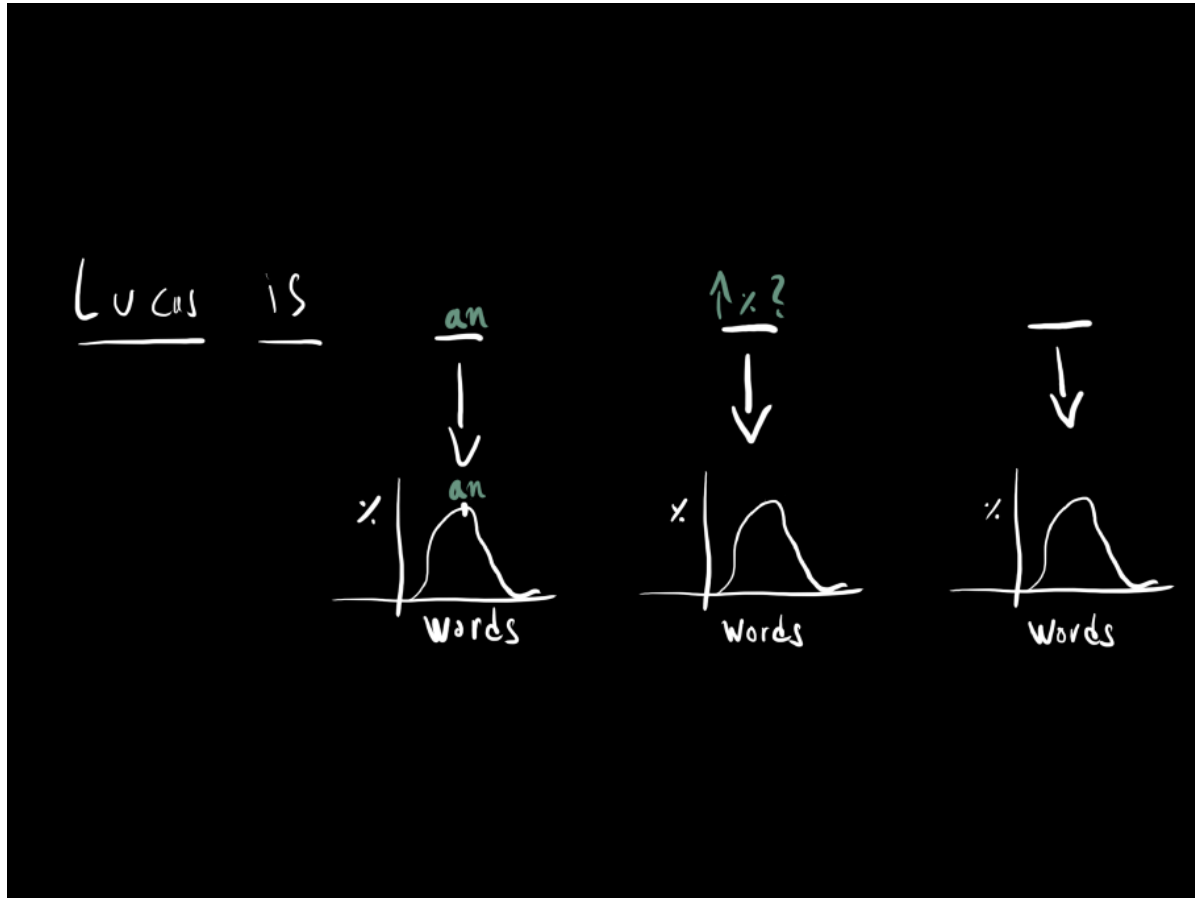
LLMs Predict the Next Word



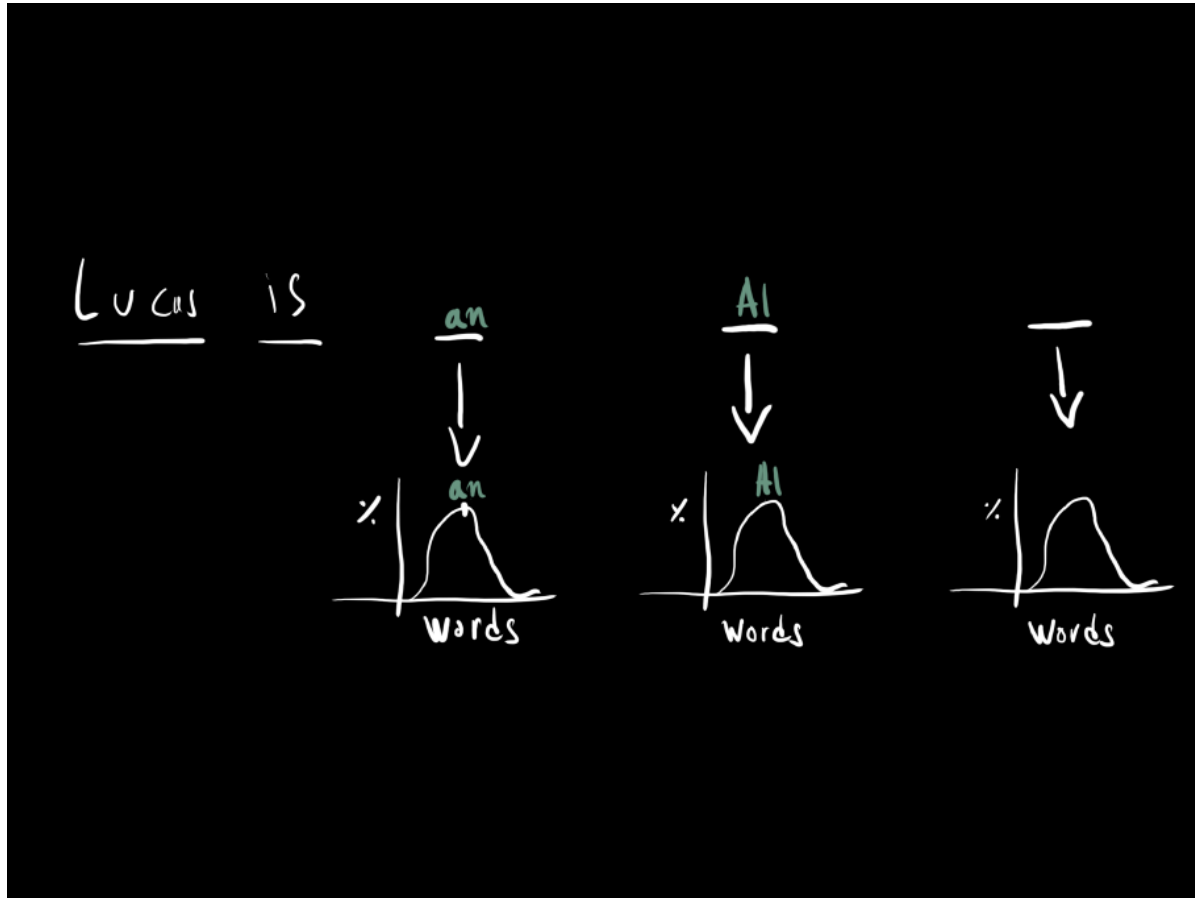
LLMs Predict the Next Word



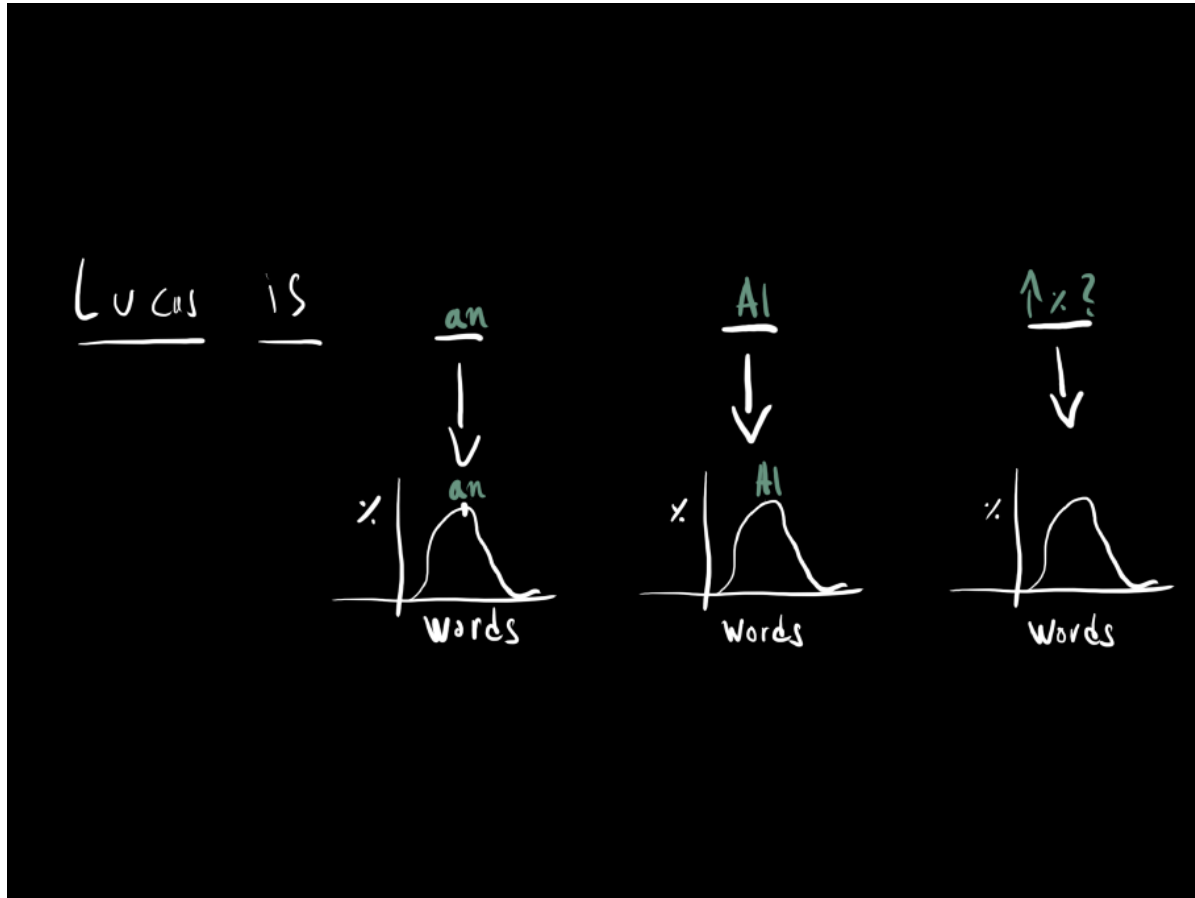
LLMs Predict the Next Word



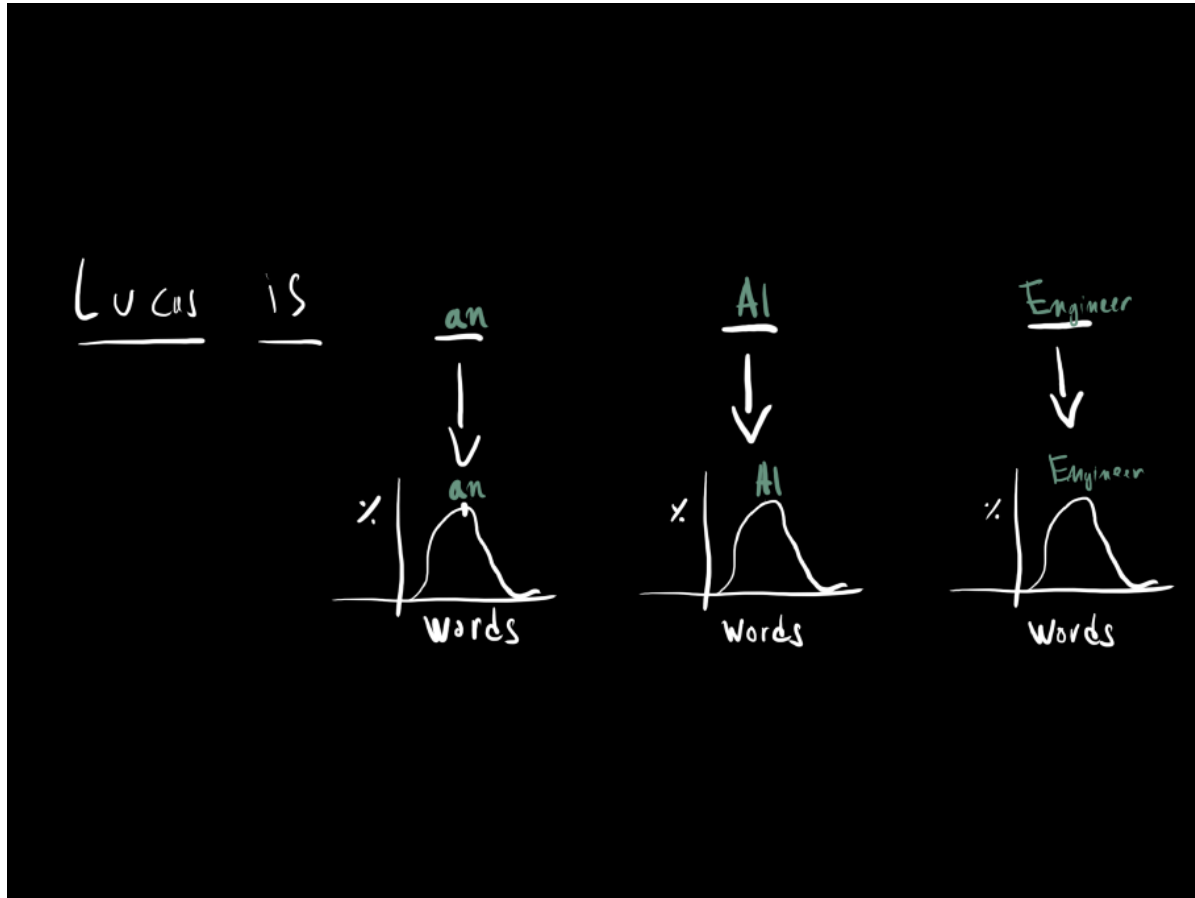
LLMs Predict the Next Word



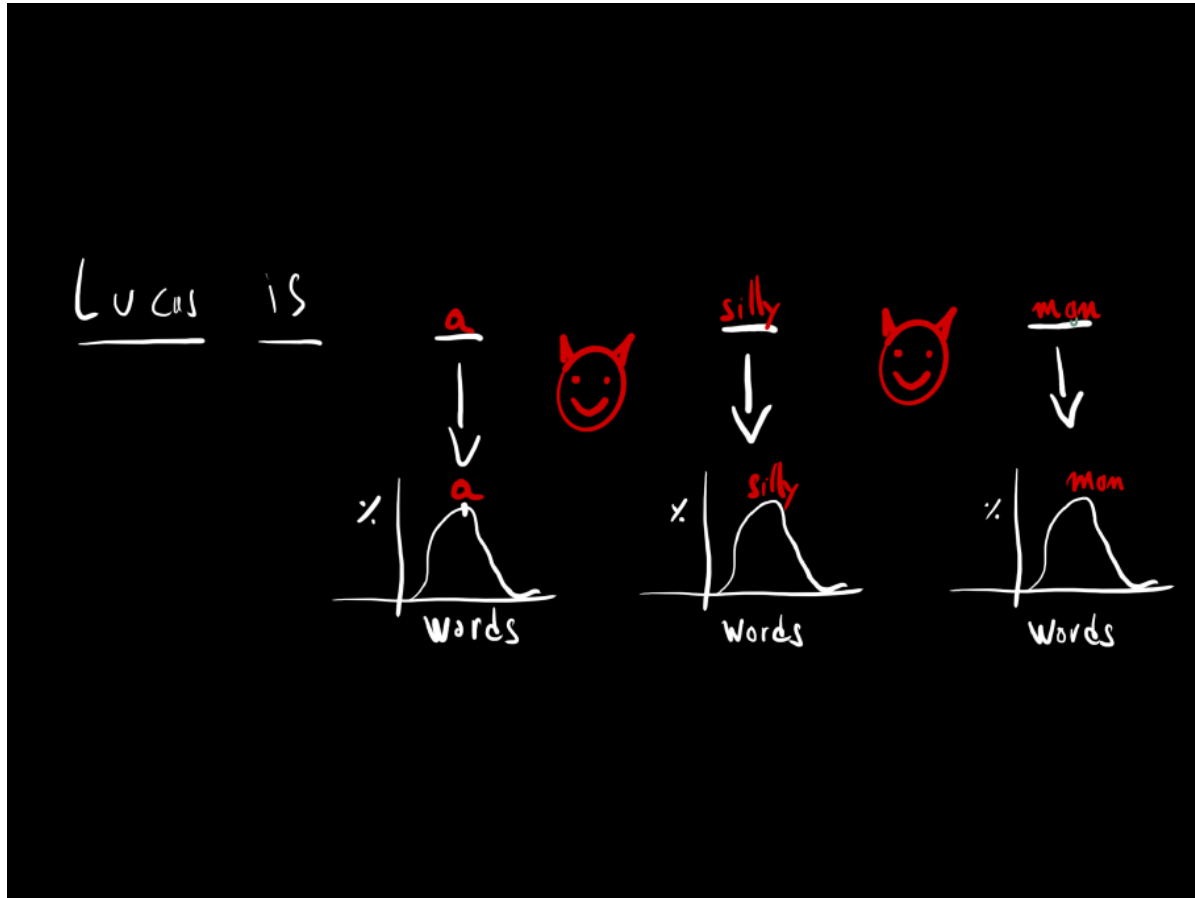
LLMs Predict the Next Word

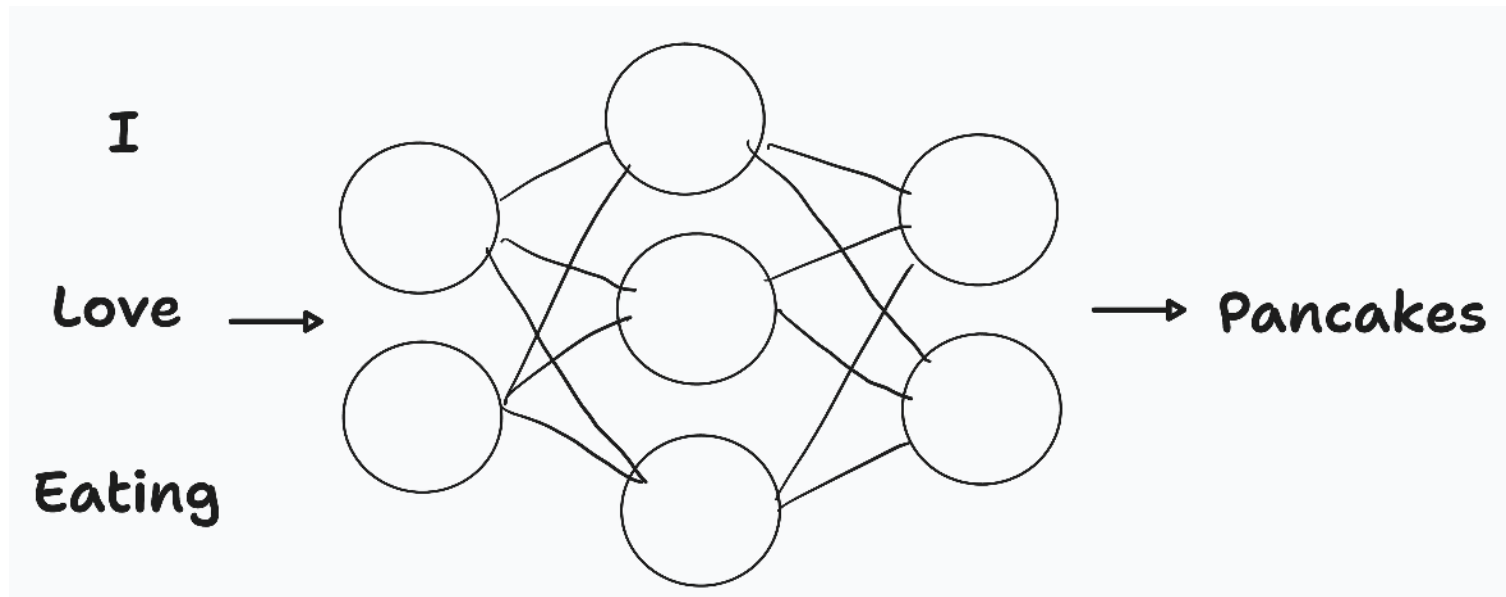


LLMs Predict the Next Word

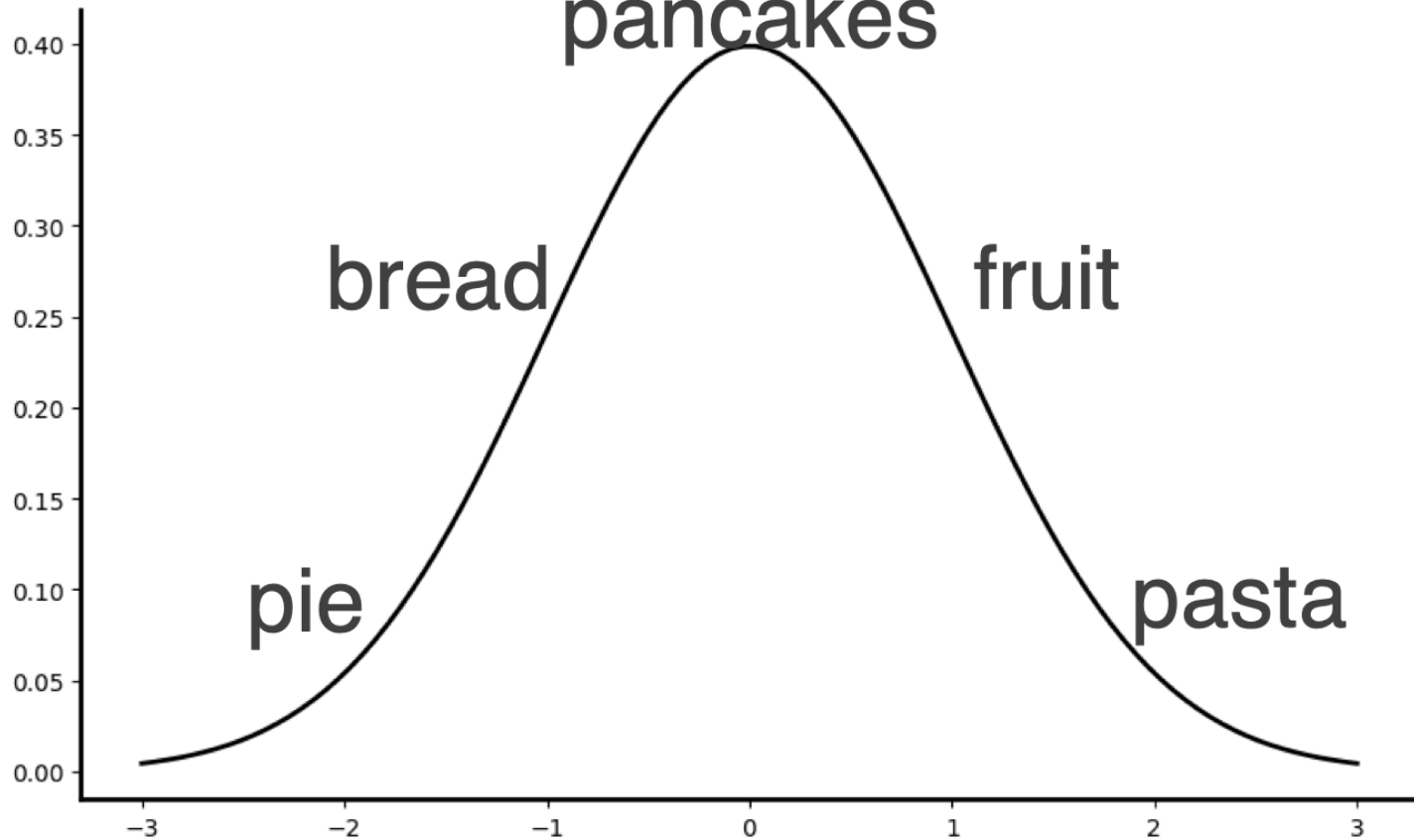


LLMs Predict the Next Word





Probability Distribution over the Next Word



Open Models: How We Got Here

Date	Model	Why It Mattered
Apr 2024	Llama 3	Open weights with a commercial license
Dec 2024	Llama 3.3 70B	Best open coding model of its generation
Jan 2025	DeepSeek-R1	Frontier-level reasoning, MIT license
Apr 2025	Llama 4 / Qwen 3	MoE architectures go mainstream
Apr 2026	Gemma 4	All compute tiers · multimodal · Apache 2.0
Aug 2026	Qwen 3.8 27B	Dense 27B, single 24 GB GPU, Apache 2.0 — local coding/agent pick

→ Gemma 4 is the **course default** from this session forward.

The 2026 Local Model Lineup

Model	RAM / VRAM	Best For	License
Gemma 4 12B	16 GB laptop / unified memory	Text + image + audio*	Apache 2.0
Gemma 4 26B (MoE)	~18 GB VRAM (19 GB download)	General + multimodal	Apache 2.0
Gemma 4 31B (dense)	20 GB VRAM	Flagship Gemma 4: general + multimodal	Apache 2.0
Qwen 3.8 27B	18 GB VRAM	Coding + agentic	Apache 2.0
Llama 3.3 70B	43 GB VRAM	Coding, instruction	Llama Community
DeepSeek-R1 32B	20 GB VRAM	Math + chain-of-thought	MIT
Phi-4 14B	9 GB VRAM	Reasoning, STEM, small GPU	MIT
Qwen3 4B	CPU only	General, fast, multilingual	Apache 2.0

Source: Ollama library, September 2026 · *audio via HF Transformers, not Ollama

Why Gemma 4 is the New Default for This Course

- **All compute tiers** — E2B/E4B run CPU-only; 12B fits 16 GB laptops; 26B MoE / 31B for 24 GB GPUs
- **Multimodal by default** — image input on every size; audio on E2B, E4B and 12B (via HF Transformers, not Ollama)
- **Clean commercial license** — Apache 2.0; no MAU caps, no EU restrictions, no license anxiety
- **Quality** — strong for its size
- **Active development** — Google iterating fast; first-class Ollama integration

```
ollama pull gemma4:26b    # 24 GB GPU
ollama pull gemma4:12b    # 16 GB laptop
ollama pull gemma4:e4b    # CPU-only
```

Whiteboard - Local LLMs Offering + Tooling Breakdown

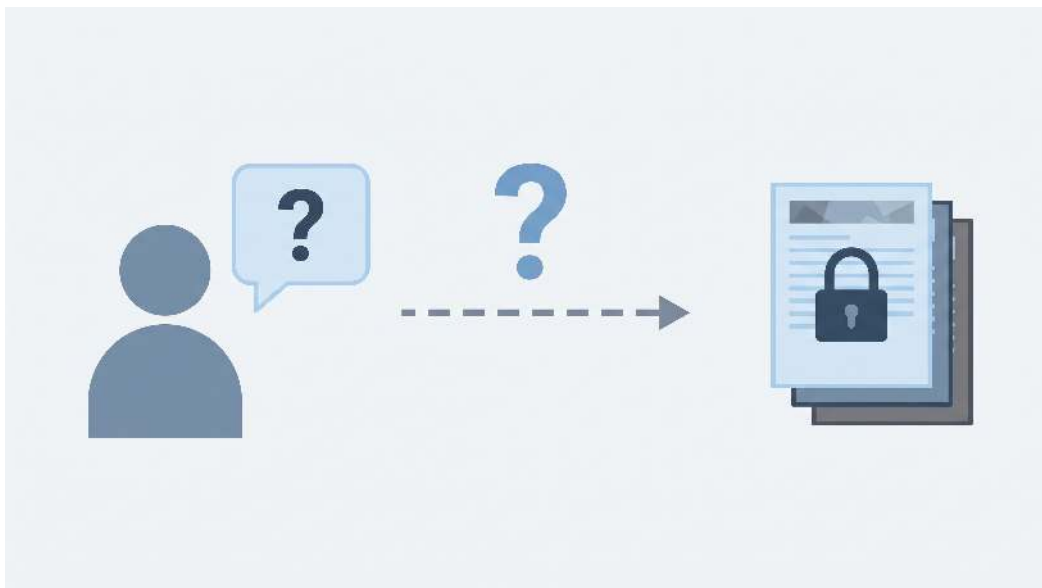
Q&A / Break

Notebook Demo - Introduction to Using Local Models Easily with Ollama

Q&A / Break

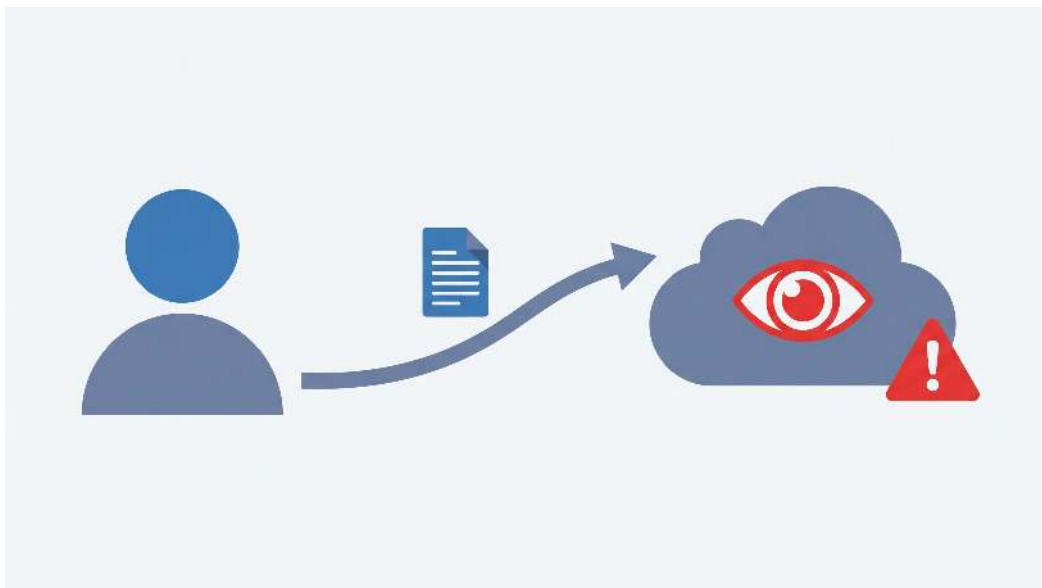
Query Your Docs Locally with Local LLMs

- Need for LLMs with access to context-relevant data.



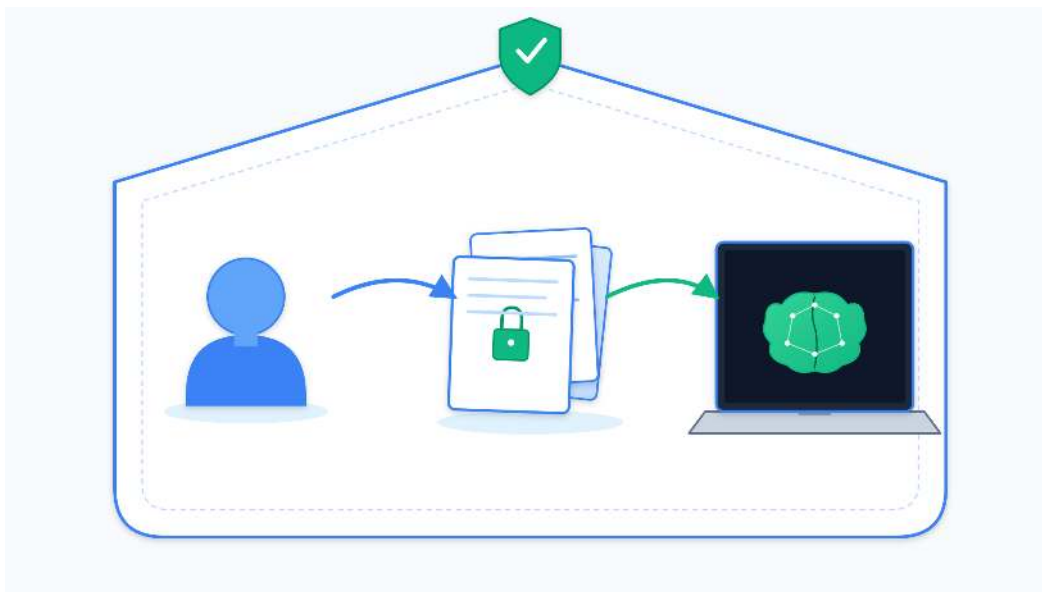
Query Your Docs Locally with Local LLMs

- Privacy concern with closed source LLMs.



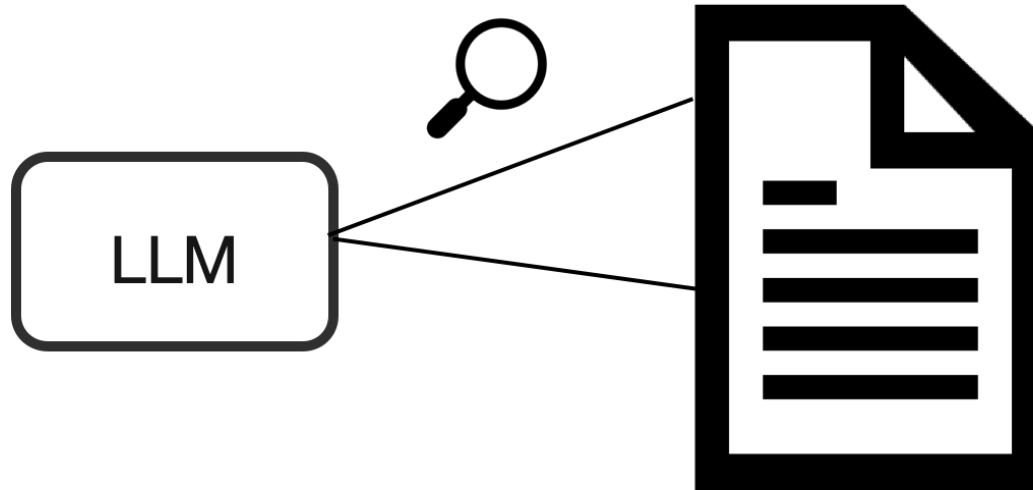
Query Your Docs Locally with Local LLMs

- Solution? Local LLMs! (like Gemma 4!)

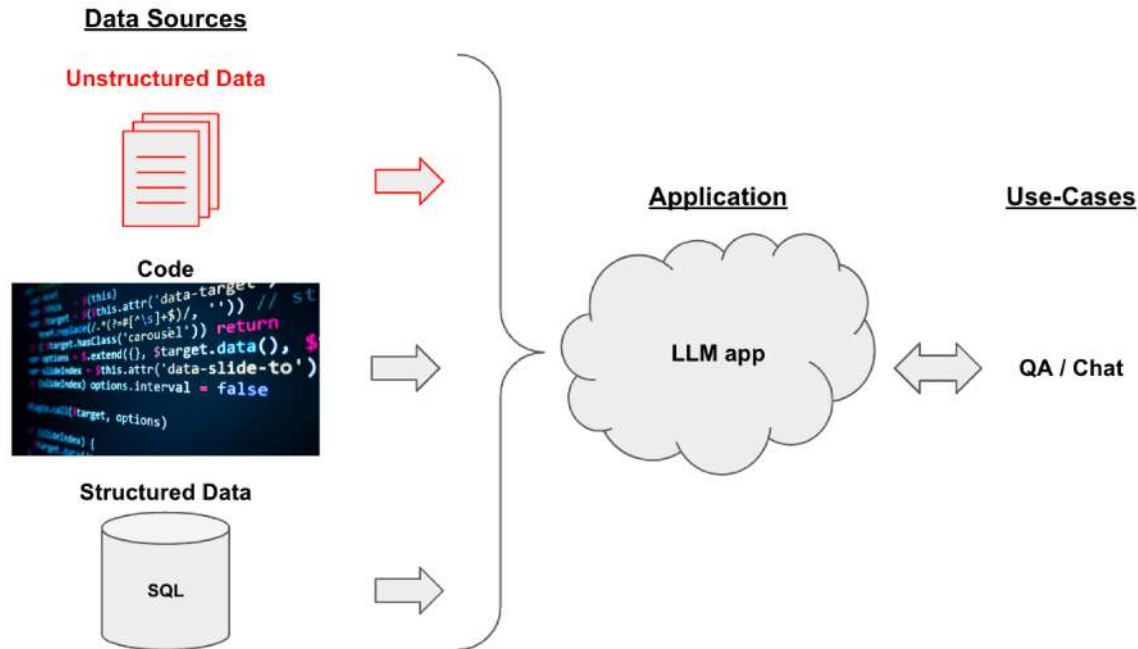


RAG with Gemma 4

- RAG - Retrieval Augmented Generation

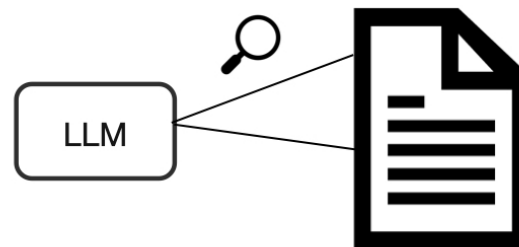


RAG with Gemma 4



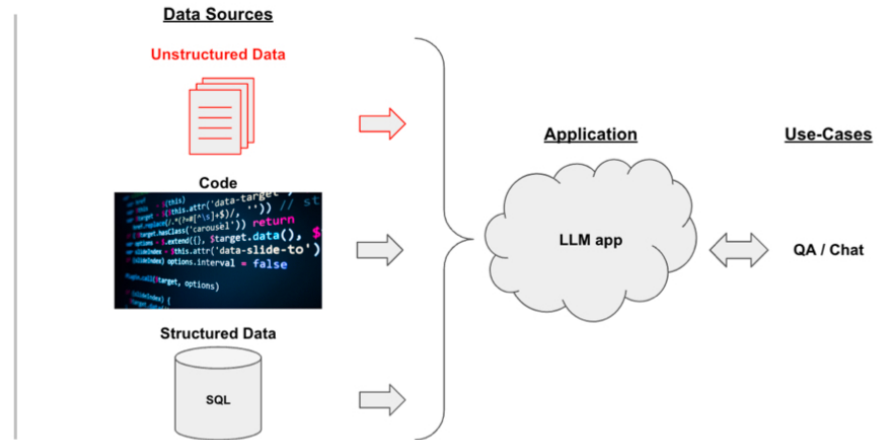


RAG - Retrieval Augmented Generation





RAG - Retrieval Augmented Generation

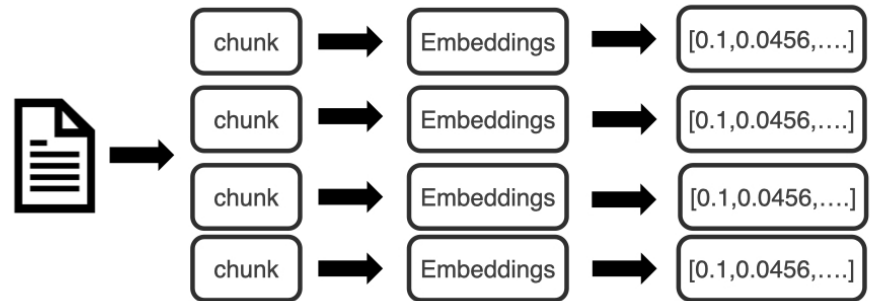


- RAG - Retrieval Augmented Generation
- LLMs have a limited context length

- RAG - Retrieval Augmented Generation

- LLMs have a limited context length

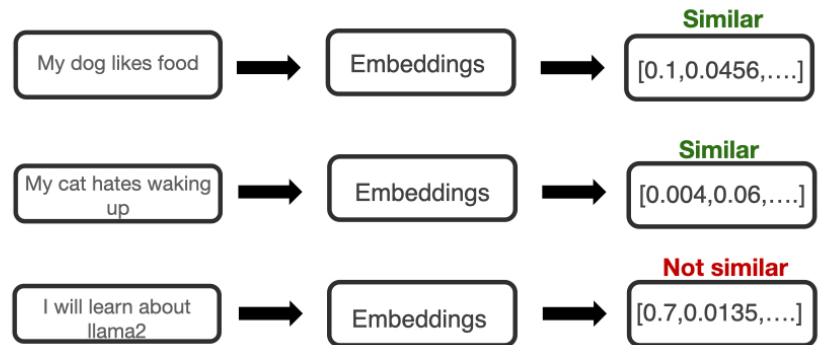
- **Embeddings:** capture content and meaning



RAG - Retrieval Augmented Generation

LLMs have a limited context length

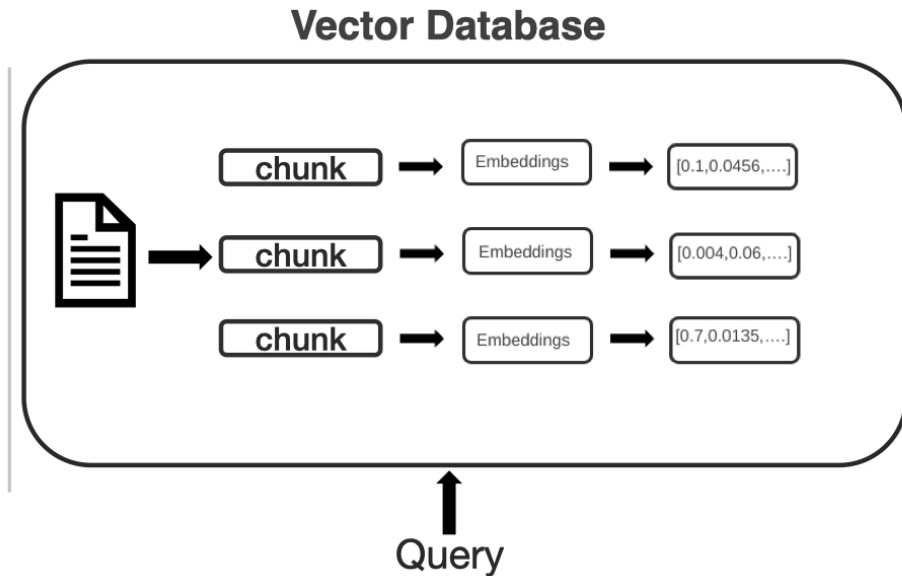
Embeddings: capture content and meaning



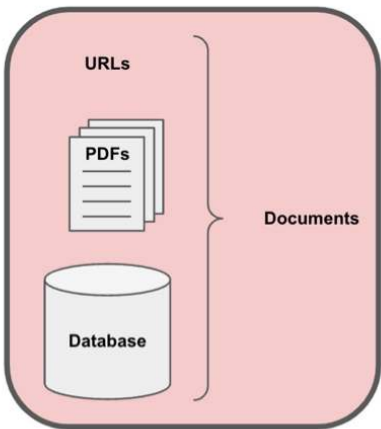
- RAG - Retrieval Augmented Generation

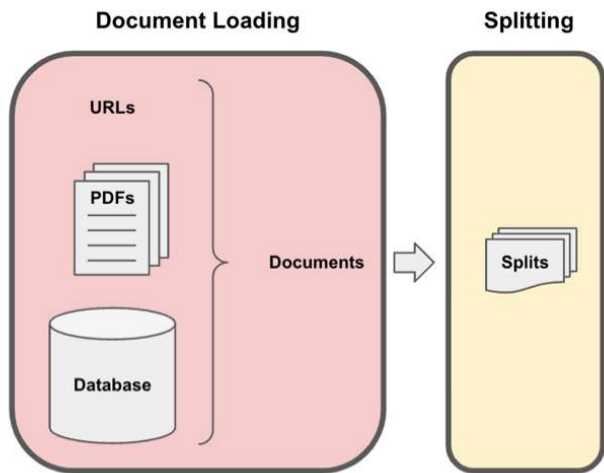
- LLMs have a limited context length

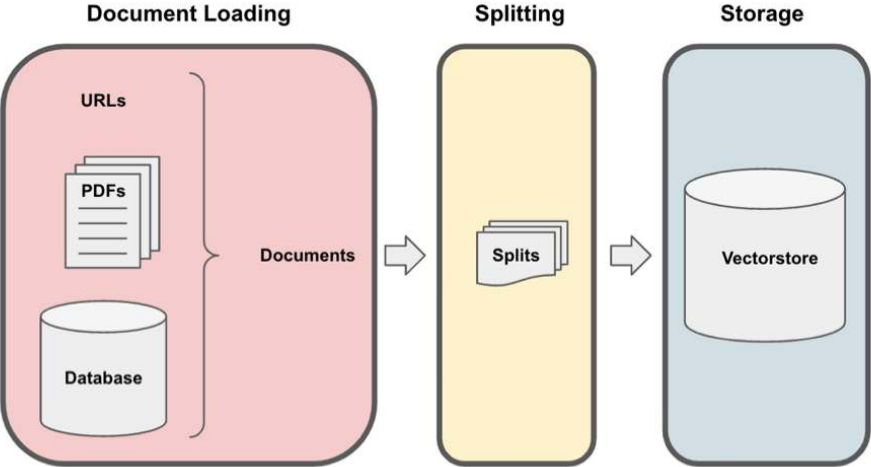
- **Embeddings:** capture content and meaning

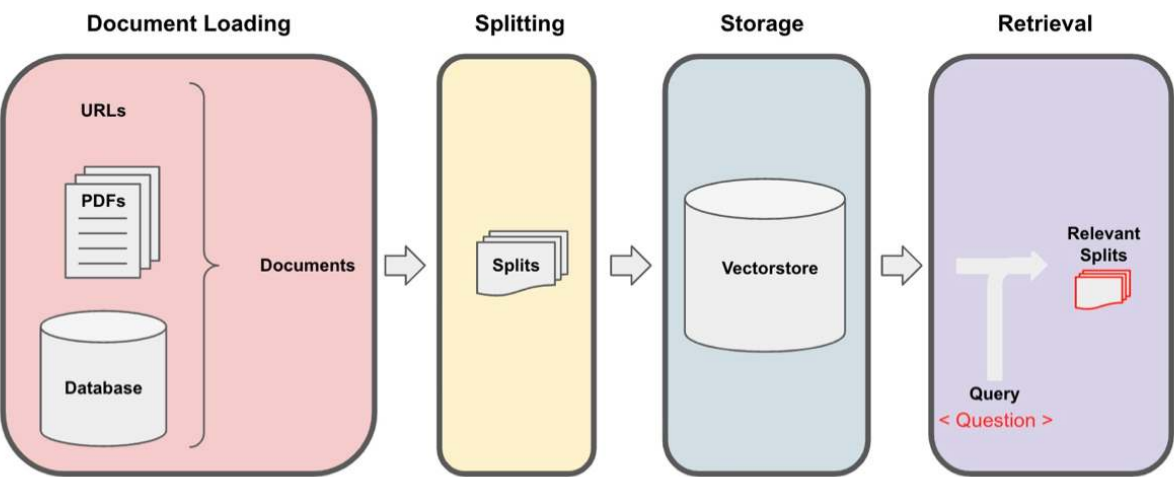


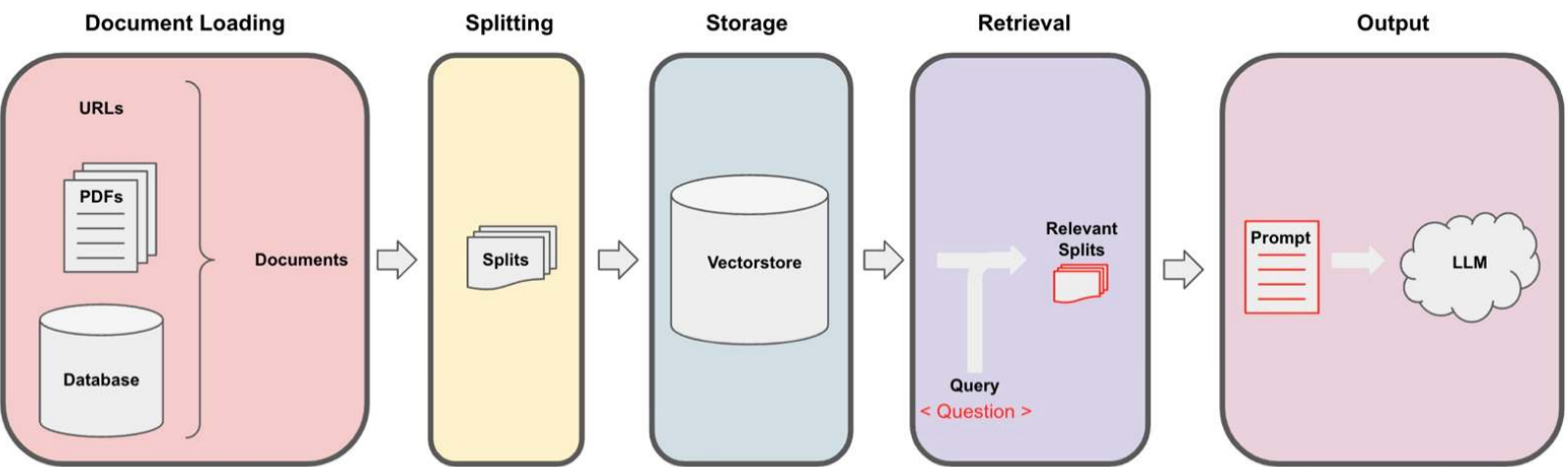
Document Loading

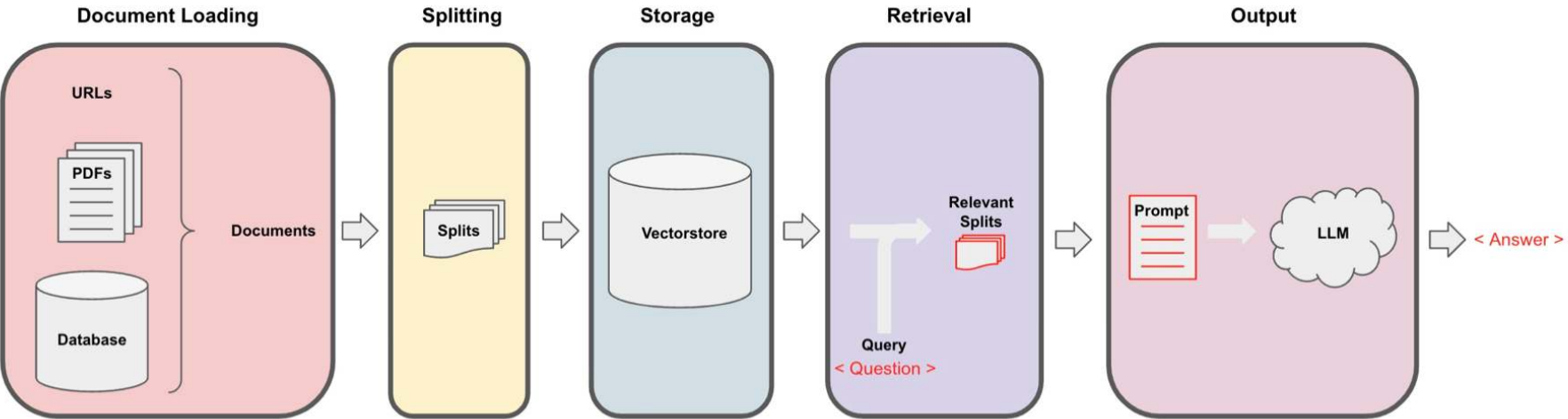












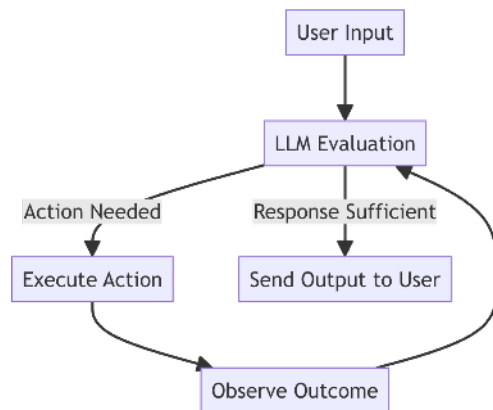
Notebook Demo - Local RAG with Gemma 4

Q&A / Break

Local Agents in 2026

What is a local agent?

- An **LLM** that receives observations and decides what to do next
- A **tool loop** — the model calls functions (search, file ops, APIs, code execution)
- **Memory** — short-term (context window) + long-term (vector store / notes)



[Explanation of the agent loop in cognitive architectures.](#)

Small Language Models for Local Agents

- SLMs (1B–14B) are now strong enough for **narrow, bounded agentic tasks**
- Classification, extraction, tool routing, short summarization → SLMs handle these well
- SLMs fail on: multi-step reasoning chains, ambiguous planning, cross-document synthesis
- **Rule of thumb:** use local SLMs for bounded execution; route to larger/cloud models for open-ended judgment
- The right architecture: **local model + retrieval + tools + eval set + fallback router**

[SLMs for Agentic Tasks — arXiv 2506.02153](#)

Hermes Agent — Full Local Agent Workbench



- Auto-detects any model running locally — no manual config per model
- **Per-model tool parsers** — correct tool calling format for every model automatically
- Bundled + self-generated skills — web search, file ops, calendar, notes, code
- Persistent memory + autonomous multi-step tasks
- One-command setup: `ollama launch hermes`

hermes-agent.nousresearch.com

Vibe Checking Local Agents

The loop:

1. Pick a small, realistic task from your use case
1. Run the agent on it with 2–3 local models
1. Score each output on 3 axes:
 - **Completion** — task done? (0 = no, 1 = partial, 2 = yes)
 - **Tool-use correctness** — right tool, right args?
 - **Hallucination** — grounded, or invented facts?
1. **Decide:** local vs. frontier
 - Local wins → use it; set up routing rule
 - Local fails → find the weak axis; route those cases to a stronger model

Build a 25-example vibe-check dataset. Score 2–3 models. Pick on evidence, not intuition.

Notebook Demo - Tool Calling/Structured Outputs and Local Agents

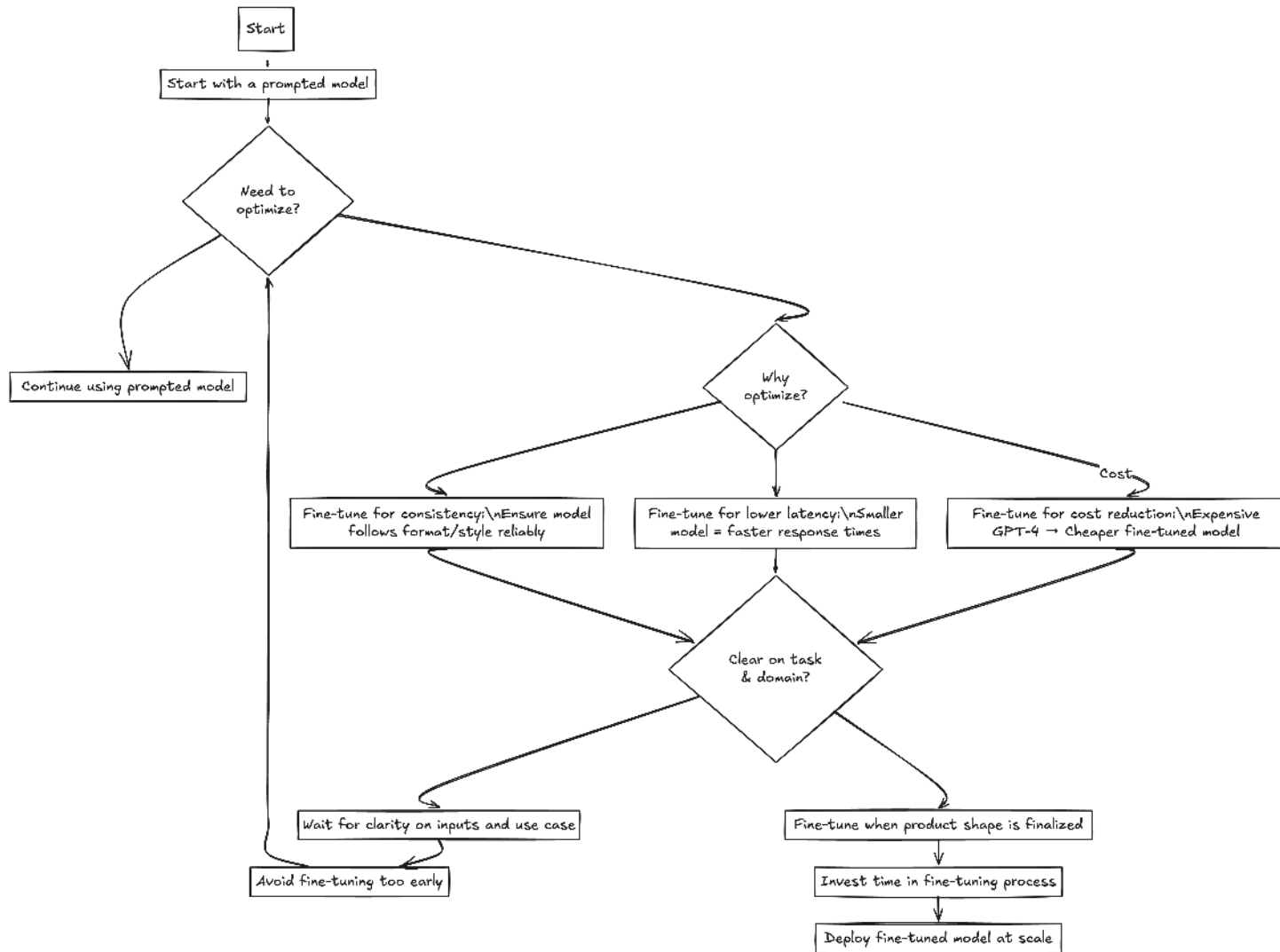
Q&A / Break

Fine Tuning Gemma 4

Whiteboard - Fine Tuning Gemma 4: what, when, why and beyond

Fine Tuning Gemma 4

- What is fine-tuning?
- It's the process of training a smaller model on a specific task leveraging a domain-specific dataset.
- What does it do?
- Fine-tuning transfers the pre-trained model's learned patterns and features to new tasks, improving performance and reducing training data needs.
- Why fine-tune?
- To improve the model's performance on a specific task.
- When to fine-tune?



Notebook Demo - Fine-Tuning Gemma 4 - Walkthrough

Q&A / Break

Notebook Demo - End To End Example with Gemma 4

References

Gemma 4

1. [Gemma 4 Model Card – Google DeepMind](#)
2. [Gemma 4 Fine-Tuning with Keras / JAX](#)
3. [Gemma 4 on Ollama](#)

RAG & Embeddings

1. [Structured Outputs with Ollama](#)
2. [Chunk Size Explanation](#)
3. [Embeddings + Parquet for Small Projects](#)
4. [Text Chunking in RAG](#)

References (cont.)

Fine-Tuning

1. [Fine Tuning Video](#)
2. [Generating Synthetic Data](#)
3. [High Quality Structured Report Generation Demo](#)

Local LLMs & Agents

1. [Awesome Local LLMs](#)
2. [LLM-Driven Data Engineering](#)

Supplementary (Llama / historical)

1. [Llama 3.3 Model Card](#)
2. [Llama Cookbook – Finetuning](#)
3. [Llama 3 Paper \(arXiv\)](#)
4. [Embeddings + Llama by Simon Willison](#)